

Experiment 5

Aim: Implement user authentication and authorization features in your mobile application, allowing users to register, log in, and access personalized content.

Theory:

User authentication and authorization are important features in mobile applications that help verify user identity and provide secure access to personalized content. Authentication allows users to register and log in, while authorization controls access to application features based on user credentials.

In this experiment, a Flutter application is developed to implement basic user authentication functionality. The application allows users to create an account, log in using their credentials, and access a personalized home screen after successful authentication. Input fields are used to collect user information, and validation ensures proper login and registration.

The application uses Flutter widgets for designing the user interface and SharedPreferences for storing user-related data locally on the device. This experiment demonstrates the implementation of secure login systems, user session handling, and personalized user interaction in mobile applications.

Source Code:

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ContactPage(),
      debugShowCheckedModeBanner: false,
    );
  }
}

class ContactPage extends StatefulWidget {
  @override
  _ContactPageState createState() => _ContactPageState();
}

class _ContactPageState extends State<ContactPage> {
  TextEditingController controller = TextEditingController();
  List<String> contacts = [];

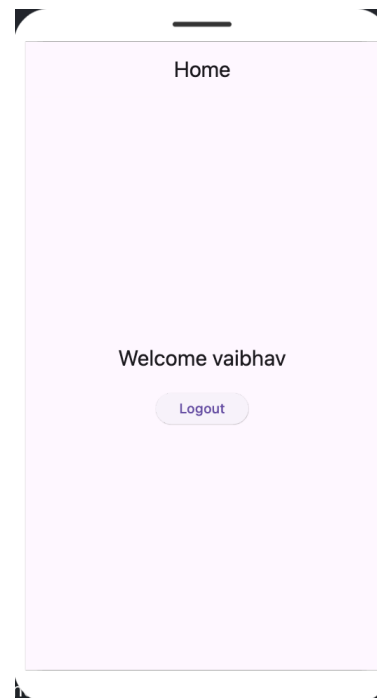
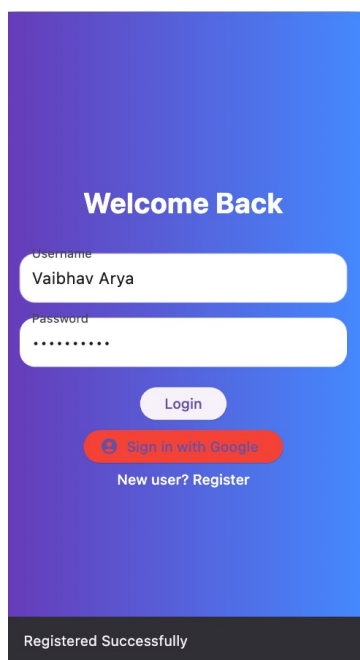
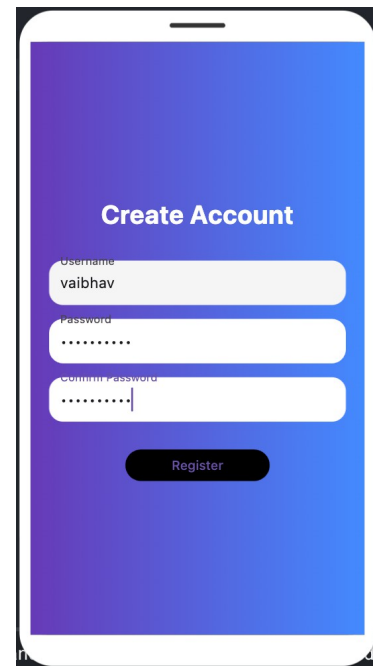
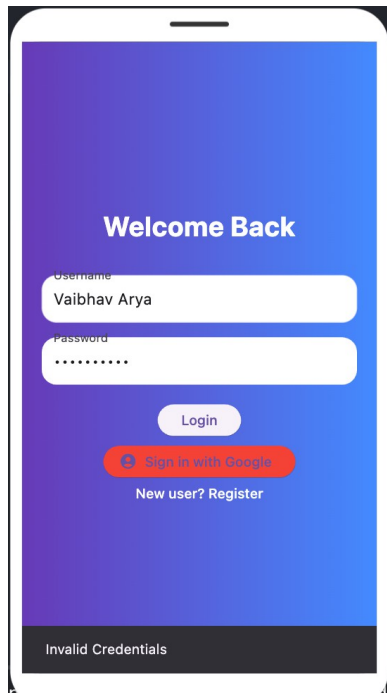
  @override
  void initState() {
    super.initState();
    loadContacts();
  }

  // Load contacts
  void loadContacts() async {
    SharedPreferences prefs = await
SharedPreferences.getInstance();
    setState(() {
```

```
        contacts = prefs.getStringList('contacts') ?? [];  
    });  
}  
  
// Add contact  
void addContact() async {  
    if (controller.text.isEmpty) return;  
  
    SharedPreferences prefs = await  
SharedPreferences.getInstance();  
  
    contacts.add(controller.text);  
  
    await prefs.setStringList('contacts', contacts);  
  
    controller.clear();  
  
    setState(() {});  
}  
  
// Delete contact  
void deleteContact(int index) async {  
    SharedPreferences prefs = await  
SharedPreferences.getInstance();  
  
    contacts.removeAt(index);  
  
    await prefs.setStringList('contacts', contacts);  
  
    setState(() {});  
}  
  
@override  
Widget build(BuildContext context) {  
    return Scaffold(  
        appBar: AppBar(title: Text("Mini Contact Saver")),  
        body: Padding(  
            padding: const EdgeInsets.all(15),
```

```
child: Column(  
  children: [  
    TextField(  
      controller: controller,  
      decoration: InputDecoration(  
        labelText: "Enter Contact Name",  
        border: OutlineInputBorder(),  
      ),  
    ),  
    SizedBox(height: 10),  
    ElevatedButton(  
      onPressed: addContact,  
      child: Text("Add Contact"),  
    ),  
    SizedBox(height: 20),  
    Expanded(  
      child: contacts.isEmpty  
        ? Center(child: Text("No Contacts Found"))  
        : ListView.builder(  
          itemCount: contacts.length,  
          itemBuilder: (context, index) {  
            return Card(  
              child: ListTile(  
                title: Text(contacts[index]),  
                trailing: IconButton(  
                  icon: Icon(Icons.delete, color:  
Colors.red),  
                  onPressed: () =>  
deleteContact(index),  
                ),  
              ),  
            );  
          },  
        ),  
    ),  
  ],  
),  
),
```

```
} ;  
}  
}
```

Output:**Learning Outcomes:**